



Instituto Tecnológico Superior de Lerdo

Ingeniería Informática

Programación Orientada a Objetos
Actividad de evaluación (ABPj) – Corte 3

ABPj: Sistema de Usuarios con Herencia en Python (un cambio de lenguaje) Propósito de la evaluación

Investigando un poco el lenguaje, el estudiante desarrollará un pequeño sistema orientado a objetos en **Python** utilizando herencia para modelar distintos tipos de usuarios de un sistema.

Además de programar y ejecutar correctamente el sistema, deberá explicar visualmente toda la solución mediante una **infografía** técnica.

Objetivo

Aplicar el concepto de **herencia** y polimorfismo creando una clase base Usuario y clases derivadas (Admin, Cliente, Invitado), reutilizando atributos y métodos.

Problema

Una plataforma digital necesita controlar distintos tipos de usuarios dentro de su sistema:

- Administradores
- Clientes
- Invitados

Todos comparten información básica, pero cada tipo de usuario tiene comportamientos y permisos distintos.

Tu tarea será desarrollar el sistema utilizando **herencia en Python**.

Requerimientos

1. Crear una clase base llamada Usuario

Atributos:

- nombre
- email
- **Métodos:**
 - mostrar_datos()
 - acceso_sistema()

2. Crear las siguientes clases derivadas

Admin

- hereda de Usuario
- agregar a la clase:
 - nivel_acceso
- y sobrescribir:
 - acceso_sistema()

Cliente

- hereda de Usuario
- agregar:
 - puntos
- sobrescribir:
 - acceso_sistema()

Invitado

- hereda de Usuario
- sobrescribir:
 - acceso_sistema()

3. Desafíos

Agregar:

- método `saludar()`
- validación de email
- menú interactivo
- lista de usuarios
- polimorfismo recorriendo usuarios en una lista

4. Aplicación principal

El programa principal deberá crear al menos:

- 1 administrador
- 1 cliente
- 1 invitado

Deberá mostrar datos de cada usuario y acceso correspondiente de cada uno.

Recuerda hacer:

- Uso correcto de clases
- Uso de herencia
- Uso de constructores (`__init__`)
- Uso de `super()`
- Métodos sobrescritos
- Código ejecutable sin errores
- Comentarios básicos en el código

5. Infografía técnica (obligatoria)

Además del programa funcionando, deberás realizar una **infografía digital** explicando todo el ejercicio y esta debe incluir:

1. Portada (datos generales)
2. Explicación del problema (lo que resuelve el sistema)
3. Diagrama de clases-herencia (UML)
4. Explicación de cada clase (atributos, métodos y propósito)
5. Capturas del código (solo fragmentos importantes)
6. Captura de ejecución (funcionando correctamente)
7. Explicación de herencia, sobreescritura de métodos, uso de `super()`.
8. Conclusiones (¿Qué aprendiste? ¿Qué fue lo más difícil? ¿Dónde podría utilizarse esto en la vida real?)

Entregables

1. Carpeta del proyecto
 - a. `usuario.py`
 - b. `admin.py`
 - c. `cliente.py`
 - d. `invitado.py`
 - e. `main.py`
2. Evidencia de ejecución (a detalle)

```

    MENÚ PRINCIPAL
1. Ver todos los usuarios
2. Saludar a todos
3. Agregar usuario
4. Salir

Elige una opción: 3

--- AGREGAR USUARIO ---
Tipo: 1) Admin 2) Cliente 3) Invitado
Selecciona tipo: 3
Nombre: fco
Email: gmail.com
Error: Email inválido: gmail.com

    MENÚ PRINCIPAL
1. Ver todos los usuarios
2. Saludar a todos
3. Agregar usuario
4. Salir

Elige una opción: 1

=====
LISTA COMPLETA DE USUARIOS
=====

[1] Admin
Nombre : Carlos Mendoza
Email : carlos@itsl.edu.mx
Nivel de acceso: 5
[Carlos Mendoza] ✓ Acceso ADMINISTRADOR - Nivel 5.
Puede gestionar usuarios, configurar el sistema y ver reportes.
=====
```

```
[1] Admin
Nombre : Carlos Mendoza
Email : carlos@itsl.edu.mx
Nivel de acceso: 5
[Carlos Mendoza] ✓ Acceso ADMINISTRADOR - Nivel 5.
Puede gestionar usuarios, configurar el sistema y ver reportes.
-----

[2] Admin
Nombre : Laura Ríos
Email : laura@itsl.edu.mx
Nivel de acceso: 3
[Laura Ríos] ✓ Acceso ADMINISTRADOR - Nivel 3.
Puede gestionar usuarios, configurar el sistema y ver reportes.
-----

[3] Cliente
Nombre : Ana Torres
Email : ana@gmail.com
Puntos acumulados: 1200
[Ana Torres] ✓ Acceso CLIENTE - 1200 puntos disponibles.
Puede ver productos, realizar compras y consultar su historial.
-----

[4] Cliente
Nombre : Luis Herrera
Email : luis@gmail.com
Puntos acumulados: 350
[Luis Herrera] ✓ Acceso CLIENTE - 350 puntos disponibles.
Puede ver productos, realizar compras y consultar su historial.
-----

[5] Invitado
Nombre : Visitante1
Email : visita1@temp.com
[Visitante1] ⚠ Acceso INVITADO - Solo lectura.
Solo puede ver contenido público. Regístrate para más funciones.
```

```
=====
SALUDOS DEL SISTEMA
=====
```

```
¡Hola, Carlos Mendoza! Bienvenido al sistema.
¡Hola, Laura Ríos! Bienvenido al sistema.
¡Hola, Ana Torres! Bienvenido al sistema.
¡Hola, Luis Herrera! Bienvenido al sistema.
Hola, Visitante1. Tienes acceso de invitado (solo lectura).
Hola, Visitante2. Tienes acceso de invitado (solo lectura).
```

```
[7] Admin
Nombre : Aguilar
Email : Aguilar@gmail.com
Nivel de acceso: 4
[Aguilar] ✓ Acceso ADMINISTRADOR - Nivel 4.
Puede gestionar usuarios, configurar el sistema y ver reportes.
-----

[9] Cliente
Nombre : pacote
Email : pacote@gmail.com
Puntos acumulados: 2000
[pacote] ✓ Acceso CLIENTE - 2000 puntos disponibles.
Puede ver productos, realizar compras y consultar su historial.
-----
```

3. Infografía
4. Respuesta a las preguntas:

a. ¿Qué ventajas ofrece la herencia en comparación con repetir código?

La herencia evita duplicar atributos y métodos comunes. En este sistema, nombre, email, mostrar_datos() y saludar() se escriben una sola vez en Usuario y las tres clases hijas los reciben automáticamente. Si mañana necesitas cambiar cómo se muestra el email, lo cambias en un solo lugar y afecta a Admin, Cliente e Invitado al mismo tiempo. Sin herencia tendrías que editar tres archivos distintos y arriesgarte a olvidar uno.

b. ¿Qué métodos fueron heredados?

Las clases derivadas heredan mostrar_datos(), saludar() y _validar_email() directamente de Usuario. El método acceso_sistema() técnicamente también se hereda, pero Admin, Cliente e Invitado lo **sobrescriben**, así que cada quien usa su propia versión en lugar de la del padre.

c. ¿Por qué sobrescribir acceso_sistema()?

Porque cada tipo de usuario tiene permisos diferentes. Un Admin necesita acceso total para gestionar el sistema, un Cliente solo puede comprar y ver su historial, y un Invitado únicamente puede ver contenido público. Si no se sobrescribiera, los tres mostrarían el mismo mensaje genérico y no habría distinción de roles, que es precisamente el problema que el sistema debe resolver.

d. ¿Qué diferencia existe entre un atributo heredado y uno propio?

Un atributo **heredado** viene del padre y está disponible en la clase hija sin necesidad de declararlo de nuevo. Por ejemplo, nombre y email los define Usuario y Admin, Cliente e Invitado los usan sin redefinirlos. Un atributo **propio** es exclusivo de esa clase y se inicializa después de llamar a `super().__init__()`. Por ejemplo, nivel_acceso solo existe en Admin y puntos solo existe en Cliente.

e. ¿Qué pasaría si todas las clases estuvieran separadas y no heredaran?

Habría duplicación masiva: cada clase tendría que declarar nombre, email, la validación de email, `mostrar_datos()` y `saludar()` por separado, repitiendo el mismo código tres veces. Además, no existiría relación formal entre las clases, lo que haría imposible el polimorfismo: el bucle `for usuario in lista_usuarios` en `main.py` funciona precisamente porque todos son Usuario. Sin herencia, tendrías que hacer tres bucles separados o usar condiciones `if` para saber de qué tipo es cada objeto, haciendo el código más frágil y difícil de mantener.

Rúbrica de evaluación

Criterio	Excelente 3	Bueno 2	Suficiente 1	Insuficiente 0
Uso de herencia	Correcto y completo	Pequeños errores	Parcial	Incorrecto
Implementación de clases	Bien estructuradas	Algunos detalles	Básicas	Incompletas
Sobrescritura de métodos	Correcta	Parcial	Limitada	Incorrecta
Ejecución del programa	Funciona completamente	Errores menores	Funciona parcialmente	No funciona
Uso de <code>super()</code>	Correcto	Parcial	Confuso	No utilizado
Organización del código	Muy clara	Clara	Regular	Desordenada
Infografía técnica	Completa y clara	Buena	Básica	Incompleta
Explicación oral/demostración	Clara y técnica	Clara	Limitada	Confusa